

CT2MAP-HN —Roadmap code/stack kỹ thuật để bắt tay implement

Mục đích tài liệu

Tài liệu này là bản roadmap triển khai kỹ thuật chi tiết cho đề tài CT2MAP-HN

theo hướng CT-only → Metabolic Heatmap + Triage, dùng như bản bổ sung/nâng cấp để đọc song song với file PDF kỹ thuật hiện có. Mục tiêu của roadmap là biến ý tưởng nghiên cứu thành một hệ thống có thể chạy được, kiểm thử được, trình diễn được, và đủ chặt chẽ để bảo vệ trước hội đồng VISEF/ISEF.

1. Nguyên tắc kiến trúc

1.1. Nguyên tắc cốt lõi

- Không sinh ảnh PET giả.
- Không tuyên bố thay thế PET/CT.
- Chỉ xây hệ thống ước lượng vùng nguy cơ chuyển hóa và điểm phân luồng ưu tiên từ CT.
- Tất cả đầu ra phải có tính giải thích tối thiểu: heatmap, lesion candidates, triage score, uncertainty score.
- Hệ thống phải có chế độ **abstain / reject option** khi độ bất định cao.

1.2. Mục tiêu triển khai

Có 4 deliverable kỹ thuật phải hình thành song song:

1. Pipeline xử lý dữ liệu 3D y sinh.
2. Baseline model đáng tin cậy.
3. Proposed model mạnh hơn baseline.
4. Dashboard demo phục vụ trình bày và chấm thi.

1.3. Chiến lược tổng thể

Thứ tự triển khai tối ưu:

1. Dựng data pipeline sạch.
2. Khóa baseline nnU-Net hoặc UNet-like trước.
3. Mở rộng sang Swin UNETR + distillation.
4. Thêm uncertainty.
5. Đóng gói inference và dashboard.

Không nên nhảy thẳng vào Swin UNETR từ ngày đầu vì bạn sẽ không biết lỗi nằm ở dữ liệu, loss, hay kiến trúc.

2. Stack kỹ thuật khuyến nghị

2.1. Ngôn ngữ và môi trường

- Python: 3.10 hoặc 3.11.
- Hệ điều hành: Ubuntu 22.04 LTS là tốt nhất; Windows dùng WSL2 nếu cần.
- Quản lý môi trường: conda hoặc mamba.
- Dependency lock: requirements.txt hoặc pyproject.toml.
- Version control: Git + GitHub private repo.

2.2. Deep learning và xử lý ảnh y sinh

- PyTorch.
- MONAI cho transforms, loaders, inferers, medical losses.
- nnU-Net v2 cho baseline rất mạnh.
- Swin UNETR qua MONAI cho proposed model.
- SimpleITK cho resampling, spacing, orientation.
- nibabel để thao tác NIfTI.
- numpy, scipy cho xử lý số.
- pandas cho manifest và metadata.
- scikit-learn cho split, AUROC, calibration, confusion matrix.

2.3. Logging, experiment tracking, reproducibility

- TensorBoard hoặc Weights & Biases.
- PyYAML cho config.
- hydra-core là lựa chọn tốt nếu muốn cấu hình nâng cao, nhưng nếu muốn đơn giản thì YAML thuần là đủ.
- rich hoặc logging tiêu chuẩn để log train/eval dễ đọc.

2.4. Demo và sản phẩm trình diễn

- Streamlit: nhanh, dễ làm, tốt cho hội đồng.
- Gradio: phù hợp nếu muốn inference flow gọn hơn.
- Plotly: biểu đồ tương tác.
- Matplotlib / seaborn: vẽ metric, calibration, ROC.

- Nếu cần viewer 3D nâng cao: ưu tiên hiển thị 2D slice viewer trước; 3D volume viewer chỉ thêm khi có thời gian.

2.5. DevOps mức học sinh nhưng sạch

- Docker cho đóng gói demo là đủ, không cần Kubernetes.
- Makefile hoặc shell scripts cho các lệnh chuẩn.
- pytest cho test các module nhỏ.

3. Phần cứng thực tế

3.1. Mức tối thiểu

- CPU 6–8 cores.
- RAM 16GB.
- GPU NVIDIA 8GB VRAM.

3.2. Mức tốt để làm ổn định

- RAM 32GB.
- GPU 12–16GB VRAM.

3.3. Chiến lược nếu máy yếu

Nếu GPU yếu, áp dụng:

- Crop ROI đầu cổ thật sớm.
- Patch-based training.
- Mixed precision (`torch.cuda.amp`).
- Gradient accumulation.
- Dùng baseline 2.5D hoặc reduced patch size trước khi lên full 3D.

4. Bài toán học máy nên đóng khung thế nào

4.1. Input

- CT đầu cổ không tiêm thuốc, đã chuẩn hóa spacing, orientation và cường độ.

4.2. Output

Đầu ra nên là bộ 4 thành phần:

1. **Metabolic Heatmap:** bản đồ liên tục giá trị nguy cơ theo voxel.
2. **Lesion Candidate Map:** mask/region đề xuất từ heatmap sau threshold + post-processing.
3. **Triage Score:** điểm nguy cơ cấp ca bệnh.
4. **Uncertainty Score:** độ bất định ở mức voxel và mức ca bệnh.

4.3. Định dạng bài toán

Mô hình chính nên được hiểu là:

- 3D voxel regression cho heatmap.
- Optional segmentation head cho lesion candidate.
- Case-level classification/regression head cho triage.

Cách đóng khung này mạnh hơn nhiều so với “sinh ảnh” vì metric đánh giá bám sát tính hữu dụng hơn.

5. Thiết kế dữ liệu

5.1. Bộ dữ liệu

Ưu tiên:

- HECKTOR 2021/2022 làm nguồn chính.
- Nếu có thể, thêm external-style validation rất nhỏ từ nguồn công khai tương thích để kiểm tra khả năng tổng quát.

5.2. Tổ chức thư mục dữ liệu

```
data/
├── raw/
│   ├── hecktor/
│   │   ├── imagesTr/
│   │   ├── labelsTr/
│   │   └── metadata/
│   ├── interim/
│   │   ├── resampled/
│   │   ├── cropped/
│   │   └── aligned/
│   ├── processed/
│   │   ├── train/
│   │   ├── val/
│   │   ├── test/
│   │   ├── manifests/
│   │   └── targets/
```

5.3. File manifest bắt buộc

Tạo `manifest.csv` với các cột sau:

- `case_id`
- `ct_path`
- `pet_path` (chỉ dùng train/teacher)
- `mask_path`
- `split`
- `site`
- `spacing_x, spacing_y, spacing_z`
- `has_primary`
- `has_nodes`
- `quality_flag`

5.4. Split dữ liệu

Đề xuất:

- Train: 70%
- Val: 10–15%
- Test: 15–20%

Quan trọng:

- Không leak bệnh nhân giữa các split.
- Nếu dataset đa trung tâm, cố gắng cân bằng theo site.
- Lưu split cố định ra file CSV/JSON để tái lập.

6. Tiền xử lý dữ liệu

6.1. Các bước bắt buộc

1. Đọc NIfTI/DICOM chuyển đổi chuẩn.
2. Chuẩn hóa orientation.
3. Resample về spacing thống nhất.
4. Windowing CT.
5. Crop ROI vùng đầu cổ.
6. Chuẩn hóa intensity.
7. Tạo target heatmap.
8. Sinh cache/processed data.

6.2. Windowing CT

Khuyến nghị soft tissue window hoặc HU clipping thực dụng, ví dụ:

- Clip trong khoảng như `[-200, 300]` hoặc range phù hợp sau khảo sát dữ liệu.
- Sau đó normalize về `[0, 1]` hoặc standardized z-score.

6.3. ROI cropping

Có 3 mức độ:

- **Mức 1:** crop theo bounding box có sẵn từ anatomy heuristics.
- **Mức 2:** crop theo vùng đầu-cổ thủ công từ metadata định nghĩa trước.
- **Mức 3:** train một anatomy locator riêng, nhưng chỉ làm khi đủ thời gian.

Đề xuất triển khai thật tế: dùng **Mức 1** hoặc **Mức 2**.

6.4. Tạo target heatmap

Có 3 kiểu target nên thử:

1. **Binary lesion mask** làm target đơn giản.
2. **Gaussian-smoothed mask** làm heatmap mềm.
3. **PET-derived soft target:** lấy vùng hấp thu cao rồi smooth thành soft map.

Khuyến nghị học sinh:

- Bắt đầu bằng Gaussian-smoothed lesion mask.
- Sau đó thêm nhánh distillation từ PET latent features.

7. Kiến trúc repo đề xuất

```
ct2map-hn/
├─ README.md
├─ requirements.txt
├─ pyproject.toml
├─ configs/
│   └─ data.yaml
│   └─ preprocess.yaml
│   └─ baseline_nnunet.yaml
│   └─ swin_unetr.yaml
│   └─ distill.yaml
│   └─ uncertainty.yaml
│   └─ demo.yaml
├─ data/
├─ notebooks/
├─ scripts/
│   └─ prepare_data.py
│   └─ build_manifest.py
│   └─ create_targets.py
```

```
| | | | | train_baseline.sh
| | | | | train_swin.sh
| | | | | eval_all.sh
| | | | | launch_demo.sh
| | | | |
| | | | | src/
| | | | | | | dataio/
| | | | | | | | | readers.py
| | | | | | | | | preprocess.py
| | | | | | | | | target_builder.py
| | | | | | | | | splits.py
| | | | | | | datasets/
| | | | | | | | | ct_dataset.py
| | | | | | | | | ct_teacher_dataset.py
| | | | | | | | | transforms.py
| | | | | | | models/
| | | | | | | | | baseline_unet.py
| | | | | | | | | swin_unetr_student.py
| | | | | | | | | teacher_encoder.py
| | | | | | | | | heads.py
| | | | | | | | | losses.py
| | | | | | | train/
| | | | | | | | | train_baseline.py
| | | | | | | | | train_swin.py
| | | | | | | | | train_distill.py
| | | | | | | | | engine.py
| | | | | | | eval/
| | | | | | | | | metrics_voxel.py
| | | | | | | | | metrics_lesion.py
| | | | | | | | | metrics_triage.py
| | | | | | | | | metrics_calibration.py
| | | | | | | | | export_figures.py
| | | | | | | inference/
| | | | | | | | | infer_case.py
| | | | | | | | | infer_batch.py
| | | | | | | | | postprocess.py
| | | | | | | demo/
| | | | | | | | | app_streamlit.py
| | | | | | | | | viewer_utils.py
| | | | | | | | | report_builder.py
| | | | | | | utils/
| | | | | | | | | seed.py
| | | | | | | | | config.py
| | | | | | | | | io.py
| | | | | | | | | logger.py
| | | | | | |
| | | | | | | outputs/
| | | | | | | | | checkpoints/
| | | | | | | | | predictions/
| | | | | | | | | metrics/
| | | | | | | | | figures/
| | | | | | | | | demo_cases/
| | | | | | |
| | | | | | | tests/
```

8. Baseline đầu tiên phải làm gì

8.1. Mục tiêu baseline

Baseline không cần quá xịn. Baseline phải trả lời được 4 câu hỏi:

- Data pipeline chạy ổn chưa?
- Loss có giảm không?
- Heatmap có học được vùng tổn thương không?
- Inference test có nhìn hợp lý không?

8.2. Baseline đề xuất

Chọn một trong hai:

- nnU-Net v2 nếu muốn baseline cực khỏe và ít phải nghĩ về kiến trúc.
- MONAI 3D UNet/BasicUNet nếu muốn dễ chỉnh code.

Khuyến nghị thật tế:

- Nếu mục tiêu nghiên cứu là so sánh với Swin UNETR, hãy dùng nnU-Net v2 làm benchmark.
- Nếu mục tiêu là học và làm chủ code nhanh, hãy dựng thêm BasicUNet nội bộ.

8.3. Loss cho baseline

Có thể kết hợp:

- Dice loss.
- BCE hoặc focal cho lesion head.
- MSE/L1 cho heatmap regression.

Một cấu hình dễ chạy:

- `total_loss = 0.5 * dice + 0.3 * bce + 0.2 * mse_heatmap`

9. Proposed model: Swin UNETR + Knowledge Distillation

9.1. Lý do chọn Swin UNETR

- Bắt ngữ cảnh dài hơn CNN thuần.
- Hữu ích khi tổn thương nguyên phát và hạch liên hệ ở khoảng xa.
- Phù hợp bài toán đầu cổ 3D có cấu trúc giải phẫu phức tạp.

9.2. Teacher-Student nên hiểu đơn giản thế nào

- Teacher nhìn CT + PET ở lúc huấn luyện hoặc đặc trưng từ PET.
- Student chỉ nhìn CT.
- Teacher ép student học đặc trưng chuyển hóa ở latent space.
- Khi triển khai thật, chỉ giữ student.

9.3. Distillation loss gợi ý

- Feature matching loss ở encoder/latent.
- Attention transfer loss nếu triển khai được.
- Output heatmap consistency loss.

Một công thức đơn giản:

- `loss_total = loss_task + lambda1 * loss_feature + lambda2 * loss_output`

9.4. Đừng over-engineer quá sớm

Không nên làm cùng lúc:

- multi-stage cascades,
- multi-teacher,
- self-distillation,
- contrastive pretraining,
- transformer quá nặng.

Đủ mạnh ở mức học sinh là:

- 1 teacher branch,
- 1 student branch,
- 1-2 loại distillation loss.

10. Uncertainty estimation

10.1. Vì sao bắt buộc

Trong đề tài này, uncertainty không phải đồ trang trí. Nó là một phần của thông điệp an toàn lâm sàng.

10.2. Cách dễ làm nhất

- Monte Carlo Dropout ở inference.
- Chạy 10-20 lần forward.
- Lấy mean prediction và variance/entropy.

10.3. Output uncertainty nên có

- Voxel-level uncertainty map.
- Case-level uncertainty score.
- Cờ cảnh báo: `LOW_CONFIDENCE_CASE = True/False`.

10.4. Luật quyết định

Ví dụ:

- Nếu triage score cao và uncertainty thấp → ưu tiên chuyển tuyến.
- Nếu triage score trung bình nhưng uncertainty cao → yêu cầu đọc lại bởi bác sĩ / không tự động khuyến nghị.

11. Thiết kế target và output

11.1. Target mức voxel

Có thể dùng:

- lesion mask,
- Gaussian heatmap,
- PET-informed saliency target.

11.2. Target mức ca bệnh

Tạo thêm các label case-level:

- `high_risk`
- `metabolically_active_case`
- `needs_priority_pet`

11.3. Pseudo-SUVmax

Chỉ nên dùng như chỉ số nội bộ ước lượng, không được trình bày như SUVmax thật.

Tên an toàn hơn:

- `Metabolic Activity Index`
- `Pseudo-Metabolic Intensity`

Khuyến nghị dùng tên đầu tiên khi đi thi.

12. Metrics đánh giá

12.1. Spatial metrics

- Dice
- IoU
- Sensitivity / Recall lesion
- Hausdorff distance nếu đủ thời gian

12.2. Heatmap/regression metrics

- MAE
- Pearson/Spearman correlation với tín hiệu mục tiêu tổng hợp
- Voxel-wise MSE

12.3. Triage metrics

- AUROC
- AUPRC
- Recall ở ngưỡng ưu tiên cao
- Specificity
- Precision
- F1

12.4. Uncertainty/calibration

- Reliability diagram
- Expected calibration error (ECE)
- Brier score

12.5. Ablation study bắt buộc

Phải có bảng ít nhất gồm:

1. CT-only baseline.
2. CT-only baseline + heatmap target mềm.
3. Swin UNETR CT-only.
4. Swin UNETR + distillation.
5. Swin UNETR + distillation + uncertainty filtering.

13. Kế hoạch code theo giai đoạn

Giai đoạn A — Dựng nền (Tuần 1-2)

A1. Tạo repo và môi trường

Checklist:

- Tạo Git repo.
- Tạo `requirements.txt`.
- Tạo `configs/`.
- Tạo cấu trúc `src/`.
- Viết `README.md` nêu cách cài, train, eval.

A2. Chuẩn hóa dữ liệu

Code cần có:

- `build_manifest.py`
- `prepare_data.py`
- `target_builder.py`

Kết quả đầu ra:

- `manifest.csv`
- các file CT đã resample/crop
- target heatmap cho train

A3. Visual sanity check

Viết notebook hoặc script xuất:

- 20 case random với CT + mask + heatmap.
- Hình giữa các slice đại diện.

Chưa có bước này thì chưa được train.

Giai đoạn B — Baseline (Tuần 3-4)

B1. Dựng dataset class

`ct_dataset.py` cần trả về:

- `image`
- `heatmap_target`
- `lesion_mask`

- `case_label`
- `meta`

B2. Dựng transform pipeline

Train transforms:

- random crop by positive region,
- flip,
- affine nhẹ,
- intensity shift nhẹ,
- gaussian noise rất nhẹ nếu cần.

Val/test transforms:

- deterministic only.

B3. Train baseline

Kết quả tối thiểu:

- loss curve,
- val metrics,
- 10 prediction cases.

B4. Eval baseline

Phải export được:

- bảng metric CSV,
- ROC figure,
- gallery best/worst cases.

Giai đoạn C — Proposed model (Tuần 5–7)

C1. Swin UNETR student

- Đảm bảo chạy được với patch size phù hợp VRAM.
- Bắt đầu không distillation trước để debug.

C2. Teacher branch

- Teacher có thể là encoder từ PET hoặc dual-input CT+PET branch trong train.
- Chỉ cần dùng teacher ở train time.

C3. Distillation

Bật từng phần:

1. output consistency,
2. feature matching,
3. attention/latent alignment nếu còn thời gian.

C4. Ablation

Không thay 5 biến một lúc. Mỗi lần chỉ thay 1 thành phần chính.

Giai đoạn D — Uncertainty + Triage (Tuần 8)

D1. MC Dropout

- Chọn các layer dropout hợp lý.
- Inference nhiều lần.
- Tính variance map.

D2. Case-level triage

Từ heatmap và lesion candidates, rút đặc trưng:

- max activation,
- volume high-risk,
- number of suspicious components,
- mean uncertainty in suspicious region.

Sau đó tính ra `triage score` bằng:

- một head classification trong model, hoặc
- logistic regression/rule-based layer bên ngoài.

Khuyến nghị học sinh: bắt đầu bằng **rule-based triage layer** từ output mô hình, sau đó mới thử head học sâu end-to-end.

Giai đoạn E — Demo (Tuần 9)

E1. Inference package

`infer_case.py` phải chạy được như sau:

```
python -m src.inference.infer_case \
  --ct path/to/case_ct.nii.gz \
  --ckpt outputs/checkpoints/best.pt \
  --out outputs/predictions/case_001/
```

E2. Dashboard

Dashboard tối thiểu có 5 vùng:

1. Upload/select case.
2. CT slice viewer.
3. Heatmap overlay.
4. Triage + uncertainty panel.
5. Download report hình ảnh.

E3. Report tự động

Sinh report HTML/PDF nhẹ gồm:

- case ID,
- representative slices,
- top suspicious regions,
- triage recommendation,
- uncertainty notice,
- disclaimer research-use only.

14. Cấu hình file config nên viết thế nào

14.1. data.yaml

Nên chứa:

- root path,
- split files,
- target type,
- spacing,
- crop size,
- HU clip,
- intensity normalization.

14.2. baseline_nnunet.yaml

Nên chứa:

- model type,
- patch size,
- batch size,

- optimizer,
- learning rate,
- scheduler,
- loss weights,
- epochs.

14.3. `swin_unetr.yaml`

Nên chứa:

- feature size,
- img size,
- dropout,
- use_checkpoint,
- patch size,
- decoder channels.

14.4. `distill.yaml`

Nên chứa:

- teacher checkpoint,
- feature layers for matching,
- loss weights,
- warmup epochs,
- freeze/unfreeze strategy.

14.5. `uncertainty.yaml`

Nên chứa:

- mc passes,
- uncertainty thresholds,
- abstain policy.

15. Skeleton lệnh chạy nên có

```
# tạo môi trường
conda create -n ct2map python=3.11 -y
conda activate ct2map
pip install -r requirements.txt

# dựng manifest
```

```
python scripts/build_manifest.py

# tiền xử lý
python scripts/prepare_data.py --config configs/preprocess.yaml

# tạo heatmap target
python scripts/create_targets.py --config configs/data.yaml

# train baseline
python -m src.train.train_baseline --config configs/baseline_nnunet.yaml

# eval baseline
python -m src.eval.export_figures --config configs/baseline_nnunet.yaml

# train swin
python -m src.train.train_swin --config configs/swin_unetr.yaml

# train distillation
python -m src.train.train_distill --config configs/distill.yaml

# batch inference
python -m src.inference.infer_batch --config configs/demo.yaml

# launch demo
streamlit run src/demo/app_streamlit.py
```

16. Những file code phải có trước tiên

16.1. Mức ưu tiên 1

- build_manifest.py
- prepare_data.py
- ct_dataset.py
- train_baseline.py
- metrics_voxel.py
- infer_case.py

16.2. Mức ưu tiên 2

- swin_unetr_student.py
- teacher_encoder.py
- train_distill.py
- metrics_triage.py
- app_streamlit.py

16.3. Mức ưu tiên 3

- `metrics_calibration.py`
- `report_builder.py`
- `pytest suite`
- `Dockerfile`

17. Pseudocode pipeline tổng quát

```
for each case:
    load CT
    standardize orientation
    resample to fixed spacing
    crop head-neck ROI
    clip HU and normalize
    build target heatmap from mask/PET-derived target

train baseline student:
    input = CT
    output = heatmap + lesion head + case score
    optimize task losses

evaluate baseline:
    compute spatial metrics
    compute triage metrics
    export figures

train proposed model:
    teacher sees PET features during train
    student sees only CT
    optimize task loss + distillation loss

inference:
    student only
    generate heatmap
    postprocess suspicious regions
    estimate uncertainty via MC dropout
    compute triage score
    render dashboard/report
```

18. Thiết kế dashboard nên thực dụng thế nào

18.1. Những gì giám khảo cần thấy

Giám khảo không cần app đẹp kiểu startup. Họ cần thấy:

- input là CT thật,
- output là heatmap có căn cứ,

- triage score rõ ràng,
- uncertainty có hiển thị,
- hệ thống có disclaimer an toàn.

18.2. Luồng UI tối thiểu

1. Chọn case.
2. Hiển thị slice axial/coronal/sagittal.
3. Overlay heatmap với opacity slider.
4. Bảng tóm tắt output.
5. Nút export report.

18.3. Nội dung khung tóm tắt

- Case ID
- Predicted high-risk regions
- Triage score
- Confidence/uncertainty
- Recommendation text
- Research-use disclaimer

19. Các lỗi phổ biến cần tránh

19.1. Lỗi dữ liệu

- split leak,
- spacing không đồng nhất,
- crop lệch anatomy,
- mask/PET misalignment.

19.2. Lỗi mô hình

- train loss giảm nhưng output vô nghĩa,
- metric tốt giả do imbalance,
- threshold chọn theo test set,
- distillation quá mạnh làm student collapse.

19.3. Lỗi trình bày

- gọi output là PET giả,
- dùng pseudo-SUV như SUV thật,
- không nói rõ uncertainty,
- nói ứng dụng lâm sàng quá mức.

20. Lộ trình 12 tuần rất cụ thể

Tuần 1

- Dựng repo.
- Tạo environment.
- Tải và kiểm tra dữ liệu.
- Viết manifest.

Tuần 2

- Viết preprocess.
- Resample/crop/windowing.
- Xuất 20 case sanity check.

Tuần 3

- Viết target builder.
- Viết dataset + transforms.
- Train baseline nhỏ để smoke test.

Tuần 4

- Train baseline thật.
- Lưu checkpoint.
- Eval spatial metrics.

Tuần 5

- Tạo case-level triage logic sơ bộ.
- Vẽ ROC/PR.
- Xuất gallery prediction.

Tuần 6

- Dựng Swin UNETR CT-only.
- So sánh với baseline.

Tuần 7

- Dựng teacher branch.
- Thêm distillation loss.
- Chạy ablation đầu tiên.

Tuần 8

- Tối ưu patch size, scheduler, loss weight.
- Chọn model tốt nhất tạm thời.

Tuần 9

- Thêm MC Dropout.
- Tính uncertainty map và case score.

Tuần 10

- Viết inference package.
- Chuẩn hóa output folder.
- Sinh case report tự động.

Tuần 11

- Dựng dashboard Streamlit.
- Tạo 5–10 demo cases đẹp.

Tuần 12

- Chạy toàn bộ pipeline từ đầu đến cuối.
- Chốt số liệu, figure, ablation table.
- Chuẩn bị slide/poster/video.

21. Mốc khóa chất lượng trước khi sang bước sau

Gate 1 — Data Ready

Chỉ được sang train khi:

- manifest sạch,
- orientation đúng,
- crop đúng,
- ít nhất 20 case đã được kiểm bằng mắt.

Gate 2 — Baseline Ready

Chỉ được sang proposed model khi:

- baseline infer ổn trên test,
- có gallery good/bad cases,
- có bảng metric CSV,
- có lỗi sai điển hình đã phân tích.

Gate 3 — Proposed Model Ready

Chỉ được sang demo khi:

- Swin/distillation tốt hơn hoặc có lý do khoa học rõ ràng,
- uncertainty hoạt động,
- output có thể giải thích.

Gate 4 — Demo Ready

Chỉ được mang đi trình diễn khi:

- app không crash,
- có 3–5 ca minh họa đẹp,
- có ca khó với uncertainty cao để chứng minh tính an toàn.

22. Thứ tự ưu tiên nếu thời gian ít

Nếu bạn chỉ có ít thời gian, ưu tiên như sau:

1. Dữ liệu sạch.
2. Baseline tốt.
3. Heatmap + triage ổn.
4. Dashboard đơn giản.
5. Uncertainty.
6. Distillation.

7. Swin UNETR full optimization.

Điều này quan trọng vì một baseline chạy chắc + câu chuyện lâm sàng tốt thường mạnh hơn một proposed model rất phức tạp nhưng chưa ổn định.

23. Bộ câu hỏi tự kiểm trước khi code tiếp

- Mỗi output có phục vụ quyết định phân luồng thật sự không?
- Có thành phần nào đang “khoe công nghệ” nhưng không giúp câu chuyện khoa học không?
- Nếu giám khảo hỏi “tại sao không dùng luôn segmentation thường?”, bạn đã có câu trả lời chưa?
- Nếu giám khảo hỏi “mô hình từ chối khi nào?”, app đã thể hiện điều đó chưa?
- Nếu giám khảo hỏi “nếu sai thì sai kiểu gì?”, bạn đã có gallery failure cases chưa?

24. Kết luận triển khai

Roadmap kỹ thuật hợp lý nhất cho đề tài này là: dữ liệu sạch → baseline nnU-Net/UNet mạnh → Swin UNETR CT-only → distillation từ PET ở train time → uncertainty estimation → triage layer → dashboard trình diễn. Toàn bộ pipeline nên được thiết kế như một **research prototype an toàn lâm sàng**, tuyệt đối tránh framing theo kiểu “thay thế PET” hoặc “sinh PET giả”.

Nếu triển khai đúng thứ tự và khóa chất lượng ở từng cổng như trên, bạn sẽ có một hệ thống vừa đủ sâu về mặt học thuật, vừa đủ thực dụng để demo, và quan trọng nhất là phù hợp với narrative an toàn – khả thi – có ích cho tuyến cơ sở.